

calc3 — Архитектура проекта

Учебный калькулятор на языке C3: токенизатор, рекурсивный спуск, AST и обход дерева визиторами.

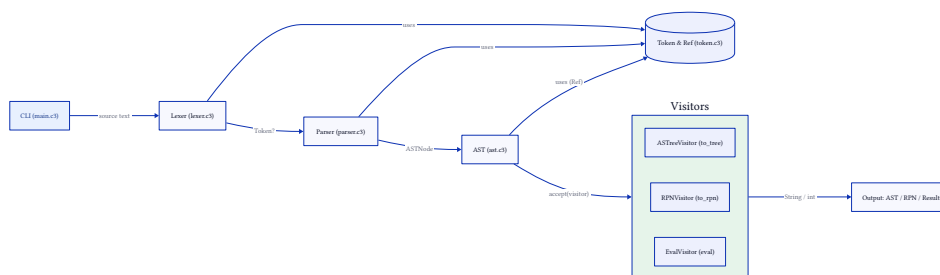
Обзор

Программа принимает текст арифметического выражения и выдаёт три представления: AST (дерево), RPN (обратная польская запись) и вычисленное значение. Поток данных — классический конвейер компилятора:

исходник -> Лексер -> Парсер -> AST --(аспет)--> Визиторы -> вывод

Все этапы используют общий тип позиции `token::Ref { row, col }`, поэтому сообщения об ошибках содержат координаты в исходнике.

Поток данных



Модули

Модуль	Назначение
token.c3	Типы токенов (TokenKind), значение токена (Value), позиция (Ref) и общий помощник форматирования ошибок <code>format_ref_error</code> .
lexer.c3	Переводит исходный текст в поток токенов (<code>Lexer.next</code>), следит за Ref, ловит неизвестные символы и переполнение целых.
ast.c3	Узлы дерева (ASTNumber, ASTUnary, ASTBinary), интерфейсы ASTNode/ASTVisitor, фабрики и демонстрационный ASTTestVisitor.
ast_tree.c3	Визитор ASTTreeVisitor + <code>to_tree</code> — вывод AST в виде дерева (в стиле eza-T).
rpn_visitor.c3	Визитор RPNVisitor + <code>to_rpn</code> — вывод в обратной польской записи (NEG/POS для унарных).
eval_visitor.c3	Визитор EvalVisitor + <code>eval</code> — вычисление значения, возвращает <code>Result{int, EvalError}</code> .
parser.c3	Рекурсивный спуск по грамматике (<code>parse</code>), построение AST, ошибки <code>ParseError</code> с позицией.
main.c3	Точка входа: разбор аргументов CLI, REPL, чтение файла и формирование итогового вывода.

Представление токенов (token.c3)

- TokenKind — перечисление: NUMBER, PLUS, MINUS, MUL, DIV, MOD, LPAREN, RPAREN, EOF, а также fault-значения UNKNOWN_CHAR, OVERFLOW.
- Token — { kind, val: Value, ref: Ref }, где Value — объединение (число или символ).

- Ref — { int row, int col }, единый источник координат ошибок.
- format_ref_error(String kind, Ref ref, String msg) — общий шаблон "<kind>(<msg> at <row>:<col>)", используется и ParseError, и EvalError.

Лексер (lexer.c3)

- Lexer хранит input, ref и pos; eat обновляет ref (в том числе перевод строки и табуляцию).
- next() -> Token?: пропускает пробелы, по switch по первому символу отдаёт токен; для цифр вызывает eat_int (с отловом переполнения). При ошибке возвращает fault (UNKNOWN_CHAR~ / OVERFLOW~), а не бросает исключение.

AST и паттерн Visitor (ast.c3)

- Интерфейс ASTNode требует accept(ASTVisitor) и to_format (для Printable).
- Интерфейс ASTVisitor — visit_number / visit_unary / visit_binary.
- Узлы: ASTNumber { ref, value }, ASTUnary { ref, op, operand }, ASTBinary { ref, op, left, right }. Каждый accept вызывает свой visit_*.
- Фабрики create_number / create_unary / create_binary аллоцируют узел и возвращают как ASTNode. op_symbol преобразует TokenKind оператора в символ.
- ASTTestVisitor — демонстрационный визитор, печатающий структуру узлов (используется в тесте).

Визиторы

Каждый визитор реализует ASTVisitor и накапливает результат в DString out (либо, для EvalVisitor, в int? value / EvalError? error):

- ASTreeVisitor → to_tree(ASTNode, Allocator) -> String — дерево с отступами и рамками.
- RPNVisitor → to_rpn(ASTNode, Allocator) -> String — постфиксная запись.
- EvalVisitor → eval(ASTNode, Allocator) -> Result{int, EvalError} — значение выражения.

Все три возвращают **владеющую** копию строки (str_view().copy(allocator)), чтобы вызывающий не держал висячую ссылку на внутренний буфер визитора.

Парсер (parser.c3)

- parse(String src, Allocator allocator) -> Result{ASTNode, ParseError} — рекурсивный спуск: expression → term → factor → primary, с учётом приоритетов и ассоциативности, поддержкой унарных +/- и скобок.
- ParseError хранит Ref и сообщение; to_format делегирует в token::format_ref_error("ParseError", ...).

Обработка ошибок

Два независимых типа ошибок, у каждого — to_format:

- ParseError — позиция и причина на этапе разбора (например, ParseError(Unexpected character at 1:4)).
- EvalError — позиция и причина на этапе вычисления (Division by zero, переполнение, включая INT_MIN / -1).

Единый token::format_ref_error гарантирует одинаковый формат и наличие координат.

Точка входа и CLI (main.c3)

Конвейер `pipeline(String src, Allocator) -> Result{PipelineOk, ParseError}` объединяет `parse + to_rpn + eval` и возвращает узел, RPN и результат за один проход. Поверх него:

- `process_line(src, allocator)` — короткий вывод RPN\результат (используется в тестах); ввод ? отвечает 42.
- `process_to_string(src, allocator)` — полный блок AST / RPN / Result.
- `process(src)` — печатает результат `process_to_string` в пуле (@pool + tmem).

Разбор аргументов командной строки вынесен в `parse_args` (заполняет состояние через `out`-параметры-указатели и возвращает, продолжать ли работу), а режимы — в `process_file` (чтение файла, разбиение на строки, обработка каждой) и `run_repl` (цикл `io::treadline` из `stdin`). `main` остаётся тонким диспетчером:

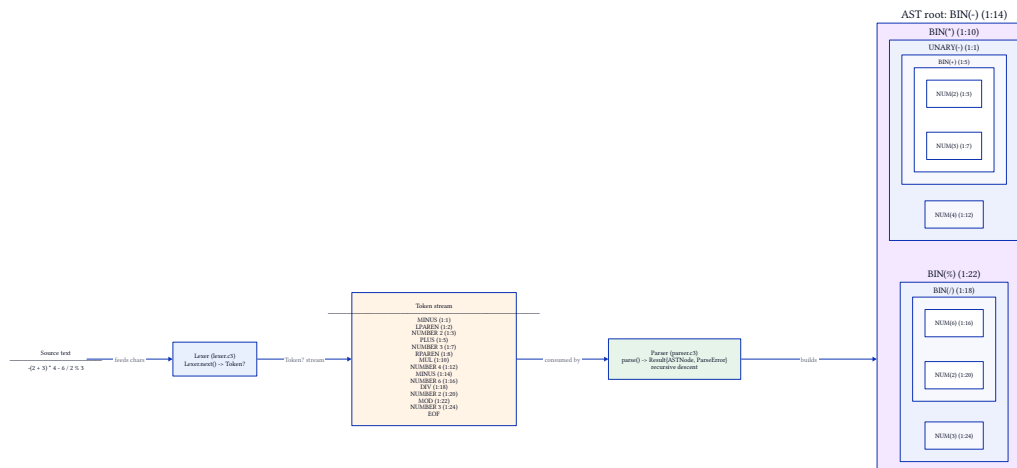
- `calc3` — интерактивный REPL;
- `calc3 "<выражение>"` — одно выражение;
- `calc3 -f <файл> / --file <файл>` — выражения из файла (по строке);
- `calc3 -h / --help` — справка;
- `--` — разделитель, чтобы передать выражение, начинающееся с `-` (например, `-5`).

Аллокатор и владение

Функции среднего уровня (`parse`, `to_tree`, `to_rpn`, `eval`, `pipeline`) принимают `Allocator` и возвращают владующие копии. Верхний уровень (`process`, `process_line`, `process_to_string`) работает в пуле `@pool` с временным аллокатором `tmem`, что упрощает освобождение памяти после формирования вывода.

Пример: от текста к AST

Разберём выражение из тестов: $-(2 + 3) * 4 - 6 / 2 \% 3$.



Лексер превращает исходник в поток токенов (каждый со своей Ref): MINUS (1:1), LPAREN (1:2), NUMBER 2 (1:3), PLUS (1:5), NUMBER 3 (1:7), RPAREN (1:8), MUL (1:10), NUMBER 4 (1:12), MINUS (1:14), NUMBER 6 (1:16), DIV (1:18), NUMBER 2 (1:20), MOD (1:22), NUMBER 3 (1:24), EOF.

Парсер (рекурсивный спуск) строит дерево:

```

BIN(-) (1:14)
├─ BIN(*) (1:10)
│   └─ UNARY(-) (1:1)
│       └─ BIN(+) (1:5)
│           ├── NUM(2) (1:3)
│           └─ NUM(3) (1:7)
└─ NUM(4) (1:12)
├─ BIN(%) (1:22)
│   └─ BIN(/) (1:18)
│       ├── NUM(6) (1:16)
│       └─ NUM(2) (1:20)
└─ NUM(3) (1:24)

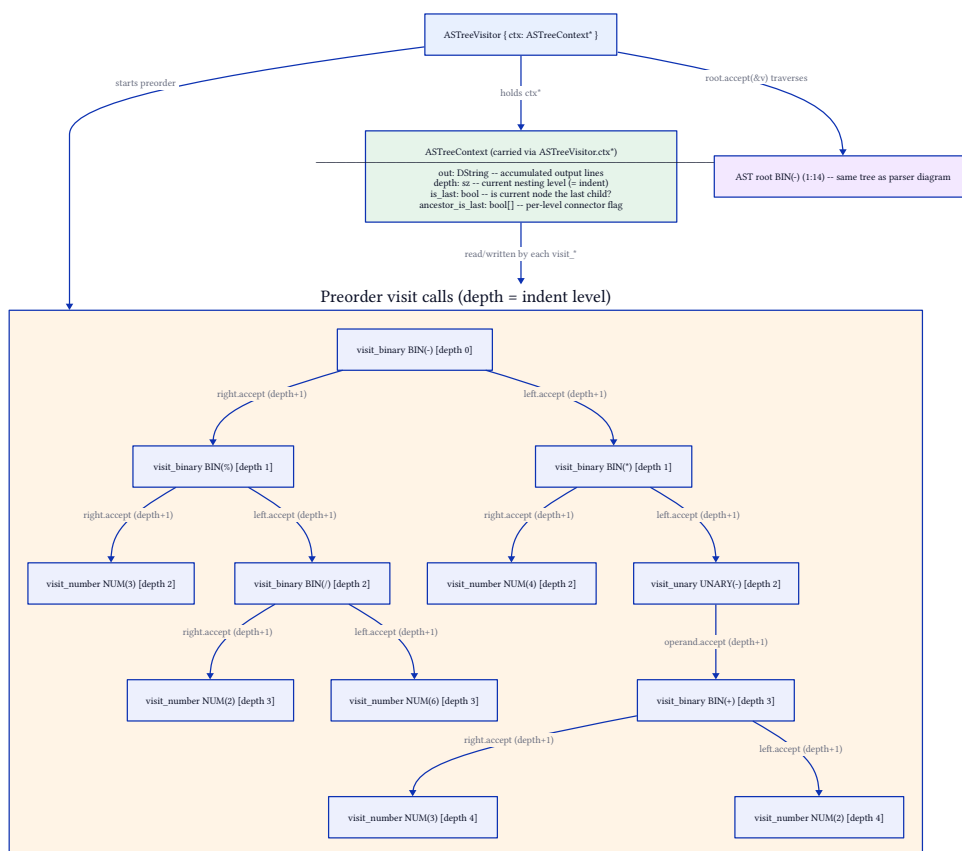
```

Координаты в узлах — это Ref соответствующих токенов; они же используются в сообщениях об ошибках.

Контекст обхода дерева (ASTreeVisitor)

to_tree создаёт ASTreeVisitor и вызывает root.accept(&v). Визитор не хранит состояние в стеке вызовов, а держит его в ASTreeContext, на который ссылается через ctx*:

- out: DString — накопленный результат (строки дерева);
- depth: sz — текущий уровень вложенности, он же отступ;
- is_last: bool — является ли текущий узел последним ребёнком родителя;
- ancestor_is_last: bool[] — для каждого предка флаг «последний ли он ребёнок», по которому выбирается соединитель | (предок не последний) или (последний).



На схеме — порядок прямого обхода (preorder). При входе в `visit_*` поле `depth` увеличивается на 1 (ребёнок рисуется на отступ глубже), при выходе — уменьшается. Флаг `is_last` выставляется перед рекурсией в детей: `false` для левого поддерева и `true` для правого, а сам узел запоминает своё значение в `ancestor_is_last[depth-1]`, чтобы дети могли нарисовать правильный префикс. Так контекст обхода сохраняется между рекурсивными вызовами без его передачи через параметры.

Сборка и тесты

```
c3c build calc3      # сборка (debug)
c3c build calc3 -O2 -g0 # release
c3c test calc3       # 74 модульных теста (лексер, AST, парсер, eval, точка входа)
```